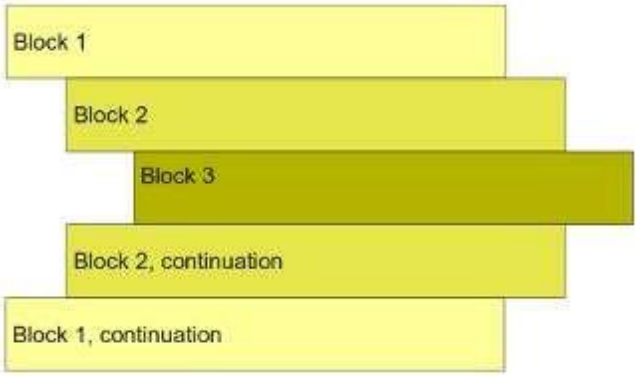
UNIT-2

Language Components

Indenting Requirements:

- Most of the programming languages like C, C++, Java use braces { } to define a block of code. Python uses indentation.
- A code block (body of a function, loop etc.) starts with indentation and ends with the first un-indented line. The amount of indentation is up to programmer, but it must be consistent throughout that block.



- Spacing out a program in this way is known as indenting. Indentation is part of the syntax of the Python language.
- Python program statements are written one above the other, as shown in the following:

```
print ("welcome")
print ("to python")
print ("Python strictly follows indentation")
```

- The 3 line program above is correct and will run producing the expected output.
- The program below will generate a syntax error because of the space character inserted at the start of the second line.

```
print ("welcome")
 print ("to python")
print ("Python strictly follows indentation")
```

- If you use the Tab key instead of using spaces for the purpose of indenting, then you must always use the Tab key.
- If you use four spaces, always use four spaces. The Python organization recommends using four spaces.

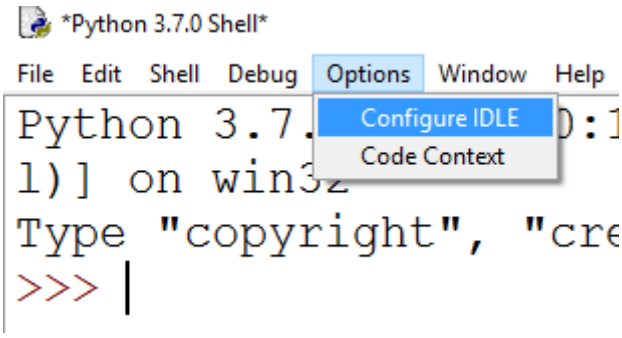
- **Example (if-else block):**

```
x = 10
if x == 10:
    print ('x has a value of 10')
else:
    print ('x does NOT have a value of 10')
```

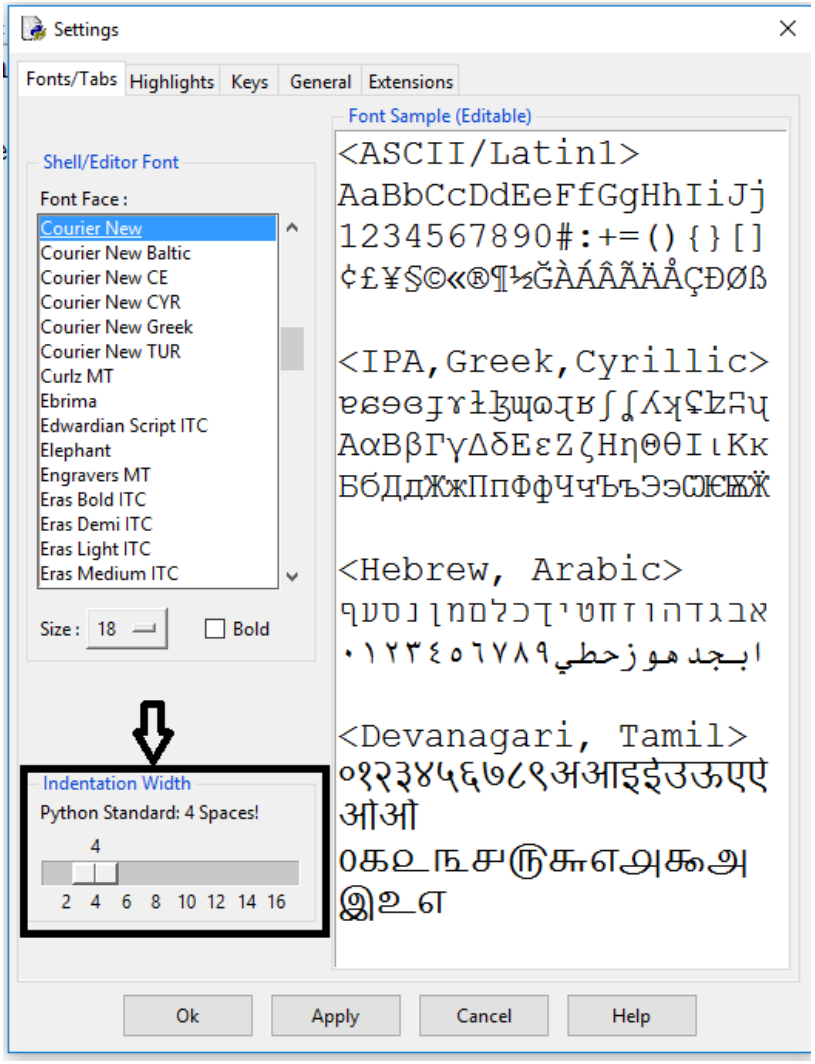
- In the example above, for indenting if and else block four space characters are used.
- **Example (function block):**

```
#function definition
def func():
    print('this is function')
    x=20

    #if-else block
    if x==20:
        print('x is equal to 20?',True)
    else:
        print('x is equal to 20?',False)

#function call
func()
```

- In the above program indentation clearly mentioned to separate block of codes.
- It is highly recommended to use only spaces, or to set the tab level to 4 on the IDLE editor options.



After clicking the Configure IDLE, Settings window will open now set indentation to 4 spaces click on apply.



- Mixing Tabs and spaces for indenting is a syntax error, even if the program looks correct on the screen! we will get an "incorrect indent" error.

****

Operators in python:

Following are the basic operators in python

- **Arithmetic operators**
- **Relational Operators**
- **Logical operators**
- **Bitwise operators**
- **Assignment operators:**
- **Special operators: Identity operators and Membership operators**

Arithmetic operators:

- Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication and division.

OPERATOR	NAME	SYNTAX
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division (float)	x / y
//	Division (floor)	x // y
%	Modulus	x % y

Example program:

```
a = 9
b = 4
add = a + b
sub = a - b
mul = a * b
# Division(float) of number
div1 = a / b
# Division(floor) of number
div2 = a // b
mod = a % b
print(add)
print(sub)
print(mul)
print(div1)
print(div2)
print(mod)
```

output:

```
13
5
36
2.25
2
1
```

Relational Operators:

- Relational operators compares the values. It either returns **True** or **False** according to the condition

OPERATOR	NAME	SYNTAX
>	Greater than	x > y
<	Less than	x < y
==	Equal to	x == y
!=	Not equal to	x != y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

- **Example Program:**

```
a = 13
b = 33

print(a > b)

print(a < b)

print(a == b)

print(a != b)

print(a >= b)

print(a <= b)
```

- **Output:**

```
False
True
False
True
False
True
```

Logical Operators:

- Logical operators perform **Logical AND**, **Logical OR** and **Logical NOT** operations.

OPERATOR	NAME	SYNTAX
And	Logical AND	x and y
Or	Logical OR	x or y
Not	Logical NOT	not x

- **Example Program:**

```
a = True
b = False

print(a and b)

print(a or b)

print(not a)
```

- **Output:**

```
False
True
False
```

Bitwise Operators:

- Bitwise operators acts on bits and performs bit by bit operation.

OPERATOR	NAME	SYNTAX
&	Bitwise AND	x & y
|	Bitwise OR	x | y
~	Bitwise NOT	~x
^	Bitwise XOR	x ^ y
>>	Bitwise right shift	x>>
<<	Bitwise left shift	x<<

- **Example:**

```
a = 10
b = 4

print(a & b)

print(a | b)

print(~a)

print(a ^ b)

print(a >> 2)

print(a << 2)
```

- **Output:**

```
0
14
-11
14
2
40
```

Assignment operators:

- Assignment operators are used to assign values to the variables.

OPERATOR	DESCRIPTION	SYNTAX
=	Assignment	x = y + z
+=	Add AND: Add right side operand with left side operand and then assign to left operand	a+=b a=a+b
-=	Subtract AND: Subtract right operand from left operand and then assign to left operand	a-=b a=a-b
*=	Multiply AND: Multiply right operand with left operand and then assign to left operand	a*=b a=a*b
/=	Divide AND: Divide left operand with right operand and then assign to left operand	a/=b a=a/b

%=	Modulus AND: Takes modulus using left and right operands and assign result to left operand	a%=b a=a%b
//=	Divide(floor) AND: Divide left operand with right operand and then assign the value(floor) to left operand	a//=b a=a//b
**=	Exponent AND: Calculate exponent(raise power) value using operands and assign value to left operand	a**=b a=a**b
&=	Performs Bitwise AND on operands and assign value to left operand	a&=b a=a&b
|=	Performs Bitwise OR on operands and assign value to left operand	a|=b a=a|b
^=	Performs Bitwise xOR on operands and assign value to left operand	a^=b a=a^b
>>=	Performs Bitwise right shift on operands and assign value to left operand	a>>=b a=a>>b
<<=	Performs Bitwise left shift on operands and assign value to left operand	a<<=b a= a<<b

Special Operators:

- **Identity Operators:**
  is and **is not** are the identity operators both are used to check if two values are located on the same part of the memory. Two variables that are equal does not imply that they are identical.

```
is          True if the operands are identical
is not      True if the operands are not identical
```

Example:

```
a1 = 3
b1 = 3
a2 = 'hello'
b2 = 'hello'
a3 = [1,2,3]
b3 = [1,2,3]
print(a1 is not b1)
print(a2 is b2)
print(a3 is b3)
```

Output:

```
False
True
True
```

- **Membership Operators:**.

in and **not in** are the membership operators; used to test whether a value or variable is in a sequence.

in	True if value is found in the sequence
not in	True if value is not found in the sequence

Example:

```
x = 'hello python'
y = [1, 4, 7]

print('G' in x)
print('hello' not in x)
print('py' in x)
print(4 in y)
print(9 not in y)
```

Output:

```
False
False
True
True
True
```

****

Conditional Statements (or) Decision Making Statements:

- Conditional statements in programming languages decides the direction of flow of program execution.
- Conditional statements are also called as Decision making statements.
  - **if statement**
  - **if-else statements**
  - **nested if statements**
  - **if-elif ladder**
- **if statement:**
  - if statement is the simple decision-making statement.
  - It is used to decide whether a certain statement or block of statements will be executed or not i.e if a certain condition is true then a block of statement is executed otherwise not.

Syntax:

```
if condition:
    #statements
        (Or)
if(condition)
    #statements
```

  - if statement accepts boolean values – if the value is true then it will execute the block of statements below it otherwise not.
  - python uses indentation to identify a block. So the block under an if statement will be identified as shown in the below example:

```
if (condition):
     #statement1
#statement2
```

Example program:

```
i = 10
if (i < 15):
     print ("10 is less than 15")
print ("I am Not in if")
```

Output:

```
10 is less than 15
```

- **if-else:**
  - The if statement alone tells us that if a condition is true it will execute a block of statements and if the condition is false it won't.
  - But what if we want to do something else if the condition is false. Here comes the *else* statement. We can use the *else* statement with *if* statement to execute a block of code when the condition is false.
  - **Syntax:**

```
if (condition):
     #statements
else:
     #statements
```

Example:

```
i = 20
if (i < 15):
      print ("i is smaller than 15")
      print ("i'm in if Block")
else:
      print ("i is greater than 15")
      print ("i'm in else Block")
print ("i'm not in if and not in else Block")
```

Output:

```
i is greater than 15
i'm in else Block
i'm not in if and not in else Block
```

- **Nested if-else:**
  - A nested if is an if statement that is the target of another if statement.
  - Nested if statements means an if statement inside another if statement. Yes, Python allows us to nest if statements within if statements. i.e, we can place an if statement inside another if statement.
  - **Syntax:**

```
if (condition1):
      if (condition2):
            #statements
      else:
            #statements
else:
      #statements
```

Example:

```
i = 20
if (i > 0):
      if (i > 15):
            print ("i is greater than 15")
      else:
            print ("i is smaller than 15")
else:
      print ("i is negative number")
```

Output:

```
i is greater than 15
```

- **if-elif-else ladder:**

The if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the ladder is bypassed. If none of the conditions is true, then the final else statement will be executed.

Syntax:

```
if (condition1):
      #statements
elif (condition2):
      #statements
else:
      #statements
```

Example:

```
i = 20

if (i == 10):
      print ("i is 10")
elif (i == 15):
      print ("i is 15")
elif (i == 20):
      print ("i is 20")
else:
      print ("i is not present")
```

Output:

```
i is 20
```

****

Looping Statements:

- Python programming language provides following types of loops to handle looping requirements.
  - **while**
  - **for**

while loop:

- A while loop implements the repeated execution of code based on a given Boolean condition.
- The code that is in a while block will execute as long as the while statement evaluates to True.
- As opposed to **for loops** that execute a certain number of times, while loops are conditionally based, so we don't need to know how many times to repeat the code going in.
- **Syntax:**

```
while (expression):
      #statements
```

- **Example:**

```
count = 0
while (count < 3):
      print("Hello Python")
      count = count+1
```

Output:

```
Hello Python
Hello Python
Hello Python
```

for loop:

- A for loop implements the repeated execution of code based on a loop counter or loop variable.
- For loops are used most often when the number of iterations is known before entering the loop, unlike while loops which are conditionally based.
- In Python, for loops are constructed like so:
- **Syntax:**

```
for iterating_var in sequence:
        statements
```

- **Example:**

```
for i in range(0,5):
        print(i)
```

Output

```
0
1
2
3
4
```

- This for loop sets up i as its iterating variable, and the sequence exists in the range of 0 to 5.
- Then within the loop we print out one integer per loop iteration. Keep in mind that in programming we tend to begin at index 0, so that is why although 5 numbers are printed out, they range from 0-4.

For Loops using range()

- One of Python's built-in immutable sequence types is range().
- In loops, range() is used to control how many times the loop will be repeated.
- When working with range(), you can pass between 1 and 3 integer arguments to it:
  1. **start** states the integer value at which the sequence begins, if this is not included then start begins at 0
  2. **stop** is always required and is the integer that is counted up to but not included
  3. **step** sets how much to increase (or decrease in the case of negative numbers) the next iteration, if this is omitted then step defaults to 1
- **Example-1:** `range(stop)`

```
for i in range(6):
        print(i)
```

Output

```
0
1
2
3
4
5
```

- **Example-2:** `range(start, stop)`

```
for i in range(20,25):
        print(i)
Output:
20
21
22
23
24
```

- **Example-3:** `range(start, stop, step)`

```
for i in range(0,10,3):
        print(i)
Output:
0
3
6
9
```

For Loops using Sequential Data Types:

- Lists and other data sequence types can also be leveraged as iteration parameters in for loops. Rather than iterating through a range(), you can define a list and iterate through that list.
- **Example:**

```
cities = ['Hyderabad', 'Vizag', 'Bangalore', 'Chennai']
for city in cities:
        print(city)

Output
Hyderabad
Vizag
Bangalore
Chennai
```

Nested Loops:

- Loops can be nested in Python, as they can with other programming languages.
- A nested loop is a loop that occurs within another loop, structurally similar to nested if statements.
- **Syntax:**

```
for iterating_var1 in outer_loop_seq: # Outer loop
        Statements # Optional
        for iterating_var in nested_loop_seq: # Nested loop
                Statements
```

- The program first encounters the outer loop, executing its first iteration. This first iteration triggers the inner, nested loop, which then runs to completion. Then the program returns back to the top of the outer loop, completing the second iteration and again triggering the nested loop.
- Again, the nested loop runs to completion, and the program returns back to the top of the outer loop until the sequence is complete or a break or other statement disrupts the process.

- **Example-1:**

```
num_list = [1, 2, 3]
alpha_list = ['a', 'b']
for number in num_list:
        print(number)
        for letter in alpha_list:
                print(letter)
```

Output:

```
1
a
b
2
a
b
3
a
b
```

- **Example-2:**

```
list_of_lists = [['a', 'i', 'm'],[0, 1, 2],[9.9, 8.8, 7.7]]
for list in list_of_lists:
        for item in list:
                print(item)
```

Output:

```
a
i
m
0
1
2
9.9
8.8
7.7
```

****

Control Statements:

- Using **for loops** and **while loops** in Python allow you to automate and repeat tasks in an efficient manner.
- But sometimes, an external factor may influence the way our program runs. When this occurs, we may want our program to exit a loop completely, skip part of a loop before continuing, or ignore that external factor. we can perform these actions with break, continue, and pass statements.

break:

- In Python, the break statement provides with the opportunity to exit out of a loop when an external condition is triggered. We will put the break statement within the block of code under loop statement, usually after a conditional if statement.

- **Example:**

```
number = 0
for number in range(10):
        number = number + 1
        if number == 5:
                break     # break here
        print('Number is ',number)
print('Out of loop')
```

- In the above program *if* the variable number is equivalent to the integer 5, *then* the loop will break.
- Within the loop is also a print() statement that will execute with each iteration of the for loop until the loop breaks, since it is after the break statement.

Output

```
Number is 1
Number is 2
Number is 3
Number is 4
Out of loop
```

- The break statement causes a program to break out of a loop.

continue:

- The continue statement gives the option to skip over the part of a loop where an external condition is triggered, but to go on to complete the rest of the loop.
- That is, the current iteration of the loop will be disrupted, but the program will return to the top of the loop.
- The continue statement will be within the block of code under the loop statement, usually after a conditional if statement.
- **Example:**

```
number = 0
for number in range(10):
        number = number + 1
        if number == 5:
                continue     # continue here
        print('Number is ',number)
print('Out of loop')
```

- The difference in using the continue statement rather than a break statement is that our code will continue despite the disruption when the variable number is evaluated as equivalent to 5.

```
Output
Number is 1
Number is 2
Number is 3
Number is 4
Number is 6
Number is 7
Number is 8
Number is 9
Number is 10
Out of loop
```

- We can use the continue statement to avoid deeply nested conditional code, or to optimize a loop by eliminating frequently occurring cases that you would like to reject.
- The continue statement causes a program to skip certain factors that come up within a loop, but then continue through the rest of the loop.

pass:

- When an external condition is triggered, the pass statement allows to handle the condition without the loop being impacted in any way; all of the code will continue to be read unless a break or other statement occurs.
- As with the other statements, the pass statement will be within the block of code under the loop statement, typically after a conditional if statement.
- **Example:**

```
number = 0
for number in range(10):
        number = number + 1
        if number == 5:
                pass     # pass here
        print('Number is ',number)
print('Out of loop')
```

Output:

```
Number is 1
Number is 2
Number is 3
Number is 4
Number is 5
Number is 6
Number is 7
Number is 8
Number is 9
Number is 10
Out of loop
```

- By using the pass statement in this program, we notice that the program runs exactly as it would if there were no conditional statement in the program. The pass statement tells the program to disregard that condition and continue to run the program as usual.
- The pass statement can create minimal classes, or act as a placeholder when working on new code and thinking on an algorithmic level before hammering out details.

Some example programs:

1. **Write a python program to print multiplication table of given number.**

Program:
```
num=int(input('Enter any value:'))
print('multiplication table of',num,'is:')
for i in range(1,11):
    print(num,'*',i,'=',num*i)
```

Output:
```
Enter any value:5
multiplication table of 5 is:
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
```

2. **Write a python program to print all even and odd numbers in given range.**

Program:
```
num=int(input('Enter any positive number:'))
even_list=[]
odd_list=[]
for i in range(0,num):
    if(i%2==0):
        even_list.append(i)
    else:
        odd_list.append(i)
print('Even numbers are:',even_list)
print('Odd numbers are:',odd_list)
```

Output:
```
Enter any positive number:20
Even numbers are: [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
Odd numbers are: [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
```

3. **Write a python program to check given number is prime or not.**

Program:
```
num = int(input('enter a positive integer value:'))
count=0
for i in range(1,num+1):
    if ((num % i) == 0):
        count+=1
if (count==2):
    print(num,'is a prime number')
else:
    print(num,'is not a prime number')
```

Output:

```
enter a positive integer value:7
7 is a prime number
```

4. Write a python program to check given number is palindrome or not.

Program:

```
n=int(input("Enter number:"))
temp=n
rev=0
while(n>0):
    rem=n%10
    rev=rev*10+rem
    n=n//10
if(temp==rev):
    print("The number is a palindrome!")
else:
    print("The number isn't a palindrome!")
```

Output:

```
Enter number:121
The number is a palindrome!
```